Synchronous Automatic Domain Randomization with Action Delays for Robust Sim-to-Real Reinforcement Learning

Alessandro Mosca Leonardo Urbani Gabriele Viceconte
Politecnico di Torino
{s333956 s344982 s349460}@studenti.polito.it

Abstract

In this work, we investigate Automatic Domain Randomization (ADR) as a method to enhance the robustness and sim-to-real transfer of reinforcement learning agents in MuJoCo locomotion environments. We propose a simple and computationally efficient variant of ADR based on a single synchronous training cycle, in which the ranges of dynamic parameters—such as masses, frictions, and action delays—are progressively expanded according to the agent's performance. Unlike OpenAI's original approach [3], our method does not require separate buffers or boundary evaluations, and allows simultaneous adaptation of multiple parameters. By including action delays, we simulate more realistic control dynamics, requiring agents to cope with both delayed inputs and varying environment conditions.

We evaluate our approach in locomotion tasks of increasing complexity, comparing ADR-trained policies with those trained in fixed environments, and our results show that ADR consistently improves robustness and stability, particularly under domain shifts, demonstrating its effectiveness in enhancing sim-to-real transfer in complex control tasks.

1. Introduction

1.1. Overview of Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make optimal decisions by interacting with an environment. At each step, the agent selects an action and receives some type of reward which is used to update its policy—function for mapping states into actions. The goal of the agent is to maximize the cumulative reward over time.

RL is particularly useful in scenarios where human intervention is costly or risky. In robotics, for example, agents can be trained in simulation or deployed in hazardous environments, such as space or underwater, reducing direct exposure of humans to danger.

1.2. The Sim-to-Real Problem

Ideally, agents would learn directly in the real environment, adapting from the start to real-world dynamics. However, this approach is often impractical: real-world learning is costly, dangerous, and time-consuming, and agents may initially perform poorly. As a result, it is common to train agents in simulation first, where the dynamics can be controlled and risks are minimized.

Despite these advantages, simulations come with limitations:

- **Rare events:** Not all possible scenarios can be captured in simulation.
- **Complex conditions:** Accurately modeling real-world physics, objects, and interactions is challenging and expensive.
- **Continuous time:** Real systems evolve continuously, while simulations often operate in discrete time steps.
- **Computational cost:** Simulations, especially in early learning stages, can require significant resources.
- **Simulation errors:** Discrepancies between simulation and reality can lead to poor real-world performance and potential damage.

1.3. Domain Randomization

To mitigate the gap between simulation and reality (the sim-to-real problem), Domain Randomization (DR) has been proposed [4]. In DR, environment parameters such as masses, frictions, or actuation delays are randomly varied within predefined ranges during training. By exposing the agent to a diverse set of simulated conditions, DR encourages the learning of policies that are robust to variations in dynamics.

The deployment of DR has several advantages, including:

- Increased likelihood that real-world parameters fall within the training distribution.
- Implicit learning to focus on the most relevant parameters, reducing sensitivity to minor variations.

- Potential emergence of adaptive behaviors resembling meta-learning when training on broad distributions.

However, DR also has drawbacks:

- The training process can be computationally expensive and require a large number of simulations;
- The selection of parameters to be randomized and their ranges is often manual and highly domain-dependent;
- Bad randomization can include physically unrealistic configurations, which can negatively impact policy optimization.

1.4. Automatic Domain Randomization

Automatic Domain Randomization (ADR) addresses some limitations of classical DR by progressively expanding the randomization ranges based on agent performance. Rather than sampling across the entire parameter space from the beginning, ADR starts with a narrow subset of environment parameters. Only when the agent reaches a performance threshold is the range expanded to include more challenging or diverse configurations.

This adaptive process allows the agent to gradually build robust behaviors, learning to handle variations in dynamics including action-delays, without being overwhelmed by extreme conditions early in training. Although initial learning may be slower due to limited exploration, ADR leads to policies that are more robust and better suited for sim-to-real transfer.

2. Related Works

Automatic Domain Randomization (ADR) has emerged as an effective strategy to reduce the sim-to-real gap in reinforcement learning by progressively increasing the variability of the environment parameters as the agent's performance improves. The core idea is to expose the agent to increasingly challenging dynamics only when it has demonstrated sufficient competence, thereby encouraging the learning of robust and transferable policies.

2.1. OpenAI's Automatic Domain Randomization

One of the most widely referenced ADR approaches is the method proposed by OpenAI in the context of dexterous manipulation [3]. In this framework, each environment parameter is associated with a dedicated buffer that stores rollouts collected at the boundaries of the current parameter range. Domain expansion is triggered only when the agent achieves satisfactory performance at these extreme values. Parameters are typically expanded one at a time following a predefined evaluation schedule.

This design provides strong warranties of robustness at the edges of the parameter space and offers an intuitive mechanism for controlled curriculum learning. However, it also introduces considerable algorithmic complexity. The use of multiple buffers, boundary-specific evaluations, and sequential parameter expansion increases the number of hyperparameters and makes the method harder to scale across environments. Moreover, treating parameters independently could neglect important interactions between physical properties, such as masses, frictions, and actuation delays.

2.2. Simplified and Online ADR Approaches

To address these limitations, several works have explored simplified ADR formulations that rely on online or synchronous updates of the randomization ranges. Instead of evaluating performance only at the boundaries, these approaches update the parameter ranges based on the average reward achieved during training episodes. Domain expansion can therefore occur at the end of each rollout, without the need for additional buffers or explicit boundary testing.

This class of methods allows multiple parameters to vary simultaneously, capturing cross-parameter interactions that are often ignored in sequential expansion schemes. Furthermore, the reduced algorithmic complexity makes these approaches more computationally efficient and easier to deploy across different tasks, as they typically rely on a small number of hyperparameters, such as a single reward threshold and a fixed expansion step.

2.3. Our Approach

In this work, we adopt this simplified, synchronous ADR paradigm and extend it to include action delays, which represent a critical source of uncertainty in real-world control systems. By jointly randomizing physical parameters (masses and frictions) and control delays, the agent is required to adapt to a richer and more realistic family of environments.

The randomization ranges are updated immediately at the end of each rollout based on a purely reward-driven criterion. This design enables faster adaptation and more efficient explorations, while avoiding the overhead of separate evaluation phases or parameter-specific buffers. As a consequence, robustness at the extreme boundaries of the parameter space is not explicitly guaranteed, and domain expansion may occur before full convergence within the current range. Nevertheless, these trade-offs are intentional and prioritize simplicity, scalability, and responsiveness.

As demonstrated in the experimental results, this approach consistently improves robustness and sim-to-real transfer performance across multiple locomotion environments, while maintaining low computational overhead and minimal tuning requirements.

2.4. Policy Training

We implemented a flexible reinforcement learning framework for MuJoCo environments using Stable-Baselines3,

supporting both standard training and Adaptive Domain Randomization (ADR). A custom ADR controller adjusts parameter ranges, based on agent performance, updating them online at the end of each rollout without extra buffers or boundary evaluations.

This simplified ADR allows simultaneous variations of multiple parameters, capturing interactions between masses, action delays, and frictions. Policies were trained with PPO and SAC (learning rate 5×10^{-4}), for 500,000 steps with default hyperparameters. Two regimes were compared:

- NO-ADR: fixed nominal environment.
- ADR: parameters are randomly sampled and progressively adapted.

2.5. Test Phase

To evaluate the computed policies, we tested the model ion different environments such as Hopper, Walker2d, Swimmer, and Ant-v5, using multiple metrics: average return, episode variability, and robustness to domain shifts. Each configuration used 10 seeds with 50 episodes per seed.

Finally, ; for Hopper and Ant, training weas repeated the training process five times to reduce stochasticity effects.

Robustness was tested in modified target environments with varied physical parameters. This setup assesses generalization beyond the training domain and highlights Automatic Domain Randomization'sDR's effect on learning stability, robustness, and sim-to-real transfer across environments of increasing dynamical complexity.

3. Results

The results are presented by environment, in order to highlight how the effectiveness of ADR depends on the complexity and structure of the underlying dynamical system.

We evaluated 4 MuJoCo-based environments with increasing levels of complexity and redundancy. For each environment, we compared different policies trained with and without ADR, using PPO, and analyzed their behavior under different conditions.

Each environment configuration was evaluated using 10 different random seeds, with 50 episodes per seed. The values reported in the tables therefore represent aggregated statistical results across these evaluations. In particular, the Hopper and Ant environments are the most significant cases; for this reason, the corresponding policies were trained 5 times each, in order to ensure that the reported results are representative and not dependent on a single training run. This follows the best practice introduced by Duan et al.[2], extended to the ADR and the *sim-to-real* context.

3.1. Hopper

The Hopper environment represents a fragile and underactuated system characterized by a very narrow stability region. Small variations in physical parameters, such as link masses, can significantly alter the robot's dynamics and easily lead to falls. This makes Hopper a particularly suitable benchmark for evaluating the robustness effects introduced by ADR.

As shown in Fig.1, the experimental results show a clear trade-off between peak performance and robustness when using Adaptive Domain Randomization, and in Fig. 2 the ADR curve (orange) is shifted to the right of the NO_ADR curve (blue), demonstrating that it is statistically superior across the entire distribution of outcomes.

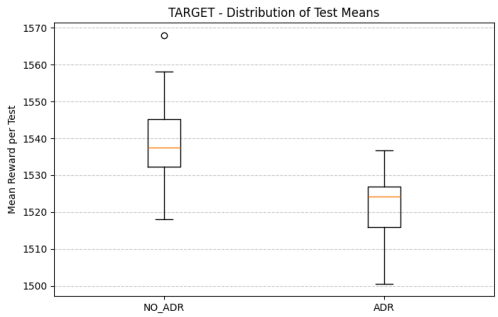

Figure 1. Reward distribution in target environment

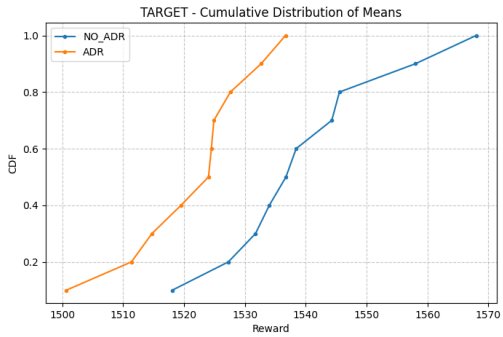

Figure 2. Cumulative Distribution Function

In the source domain, the policy trained without ADR achieves higher performance than the policy trained with ADR. This indicates that, in a fixed and nominal environment, the non-ADR policy can specialize its behavior more effectively and exploit the exact dynamics of the simulator. However, the situation changes in the target domain, where the "Torso" mass is shifted by +1 kg. Here, the ADR-trained policy achieves slightly higher mean performance and, more importantly, exhibits significantly lower variance. This indicates that ADR produces a more stable and reliable policy when faced with unseen dynamics. These results highlight a key property of ADR in underactuated systems such as Hopper: ADR intentionally sacrifices some performance of the source domain while improving

stability and robustness under domain shift. Rather than optimizing for a single set of dynamics, the policy learns behaviors that remain effective across a broader range of physical conditions.

Table 1. Robustness statistics in source and target environments for Hopper.

Env.	Method	Mean $\pm$ Std	CV	P10
Source	NO_ADR	1727.7 ± 3.8	0.002	723.2
	ADR	1467.0 ± 2.6	0.002	1464.4
	SAC	$\mathbf{1614.6 \pm 17.5}$	**0.011**	–
Target	NO_ADR	1540.2 ± 13.8	0.009	1526.4
	ADR	1521.7 ± 10.0	0.007	1464.4
	SAC	$\mathbf{1420.7 \pm 96.2}$	**0.068**	–

Table 1 reports robustness statistics in both source and target environments. For each episode return, we report its average and standard deviation together with the coefficient of variation (CV), minimum return, and P10.
Unless NO_ADR achieves competitive performance in the source domain, it exhibits large variance and severe degradation under domain shift.ADR significantly reduces variability and improves worst-case performance, while SAC provides the highest overall robustness in both domains.

3.2. Walker2d

Walker2d is a more constrained and structurally simple environment compared to Hopper. Although it is underactuated, its dynamics allow for a relatively stable walking strategy. As a result, learning in Walker2d is expected to be generally easier and less sensitive to moderate variations in physical parameters.

Table 2. Robustness statistics in source and target environments for Walker.

Env.	Method	Mean $\pm$ Std	CV	P10
Source	NO_ADR	1606.3 ± 35.9	0.037	949.4
	ADR	1327.7 ± 70.5	0.053	1235.0
Target	NO_ADR	980.2 ± 98.5	0.061	1496.5
	ADR	1461.1 ± 56.1	0.038	1388.4

The results in Table 2 show the same behavior as in the Hopper experiments. Specifically, the NO_ADR policy achieves higher performance in the source domain, but suffers a drastic drop in the target environment where its coefficient of variation nearly doubles. In contrast, the ADR-trained policy demonstrates superior generalization; while it starts with a lower mean in the source domain, it actually improves its performance in the target domain. Most importantly, ADR maintains a much higher 10th percentile compared to the one of NO_ADR in the source (which falls significantly in the target), confirming that ADR forces the agent to learn a more conservative and reliable strategy that is less susceptible to failure when the environment's physics are perturbed.

3.3. Swimmer

The Swimmer environment differs significantly from the other locomotion tasks considered in this study: the agent operates in a low-dimensional environment with highly constrained dynamics, and the reward is lower than in other environments.

In our task, the positive reward incentivizes forward velocity, and for this reason testing it shifting some masses or with ADR did not produce any notable effect. Furthermore, the lower standard deviation of the test results (Table 3) indicates that this task is not very informative, making this environment not adapt to be tested with ADR algorithm. For completeness, we report the test results in Table 3.

Table 3. Robustness statistics in source and target environments.

Env.	Method	Mean $\pm$ Std	CV	P10
Source	NO_ADR	183.8 ± 0.4	0.002	183.4
	ADR	74.5 ± 0.2	0.002	74.2
Target	NO_ADR	187.9 ± 0.3	0.002	187.4
	ADR	74.6 ± 0.2	0.002	74.4

3.4. Ant-v5

Ant represents the most complex environment considered in this work. It features a high-dimensional state and action space, significant redundancy in actuation, and highly nonlinear dynamics. These characteristics result in a challenging optimization landscape with many local minima and a wide variety of possible locomotion strategies.

The results reported in Table 4 demonstrate that ADR leads to higher average reward values compared to training without ADR, not only in the target domain, but also in the source domain. This behavior may appear counterintuitive if ADR is interpreted exclusively as a strengthening technique, but this interpretation is incomplete. We noticed that ADR substantially modifies the distribution of environments visited during training. The agent no longer interacts with a single fixed environment, but with a family of closely related MDPs characterized by progressive variations in dynamic parameters [1].

In a highly redundant and non-linear environment like Ant, this variation forces the policy to avoid fragile, highly specialized solutions and instead encourages the research of more stable and globally effective strategies. Rather

than making the task easier, the parameter variations prevent the optimization algorithm from prematurely converging to suboptimal local minima. In this regime, the primary observable benefit is not the increase in inter-domain robustness, but rather an improvement in the quality of the learned policy due to more favorable learning dynamics.

Table 4. Robustness statistics in source and target environments.

Env.	Method	Mean $\pm$ Std	CV	P10
Source	NO_ADR	527.8 ± 35.0	0.066	491.6
	ADR	720.5 ± 24.5	0.034	689.8
	With Delay	306.8 ± 15.7	0.051	285.2
Target	NO_ADR	525.3 ± 50.0	0.095	465.0
	ADR	704.5 ± 37.9	0.054	655.5
	With Delay	299.7 ± 20.6	0.069	274.0

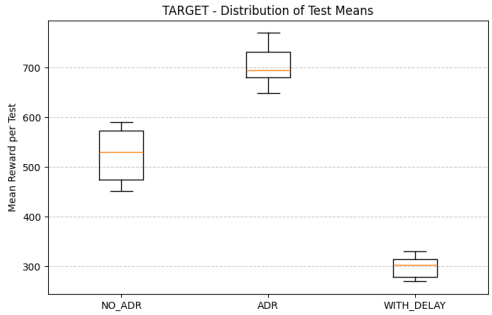

Figure 3. Reward distribution in target environment

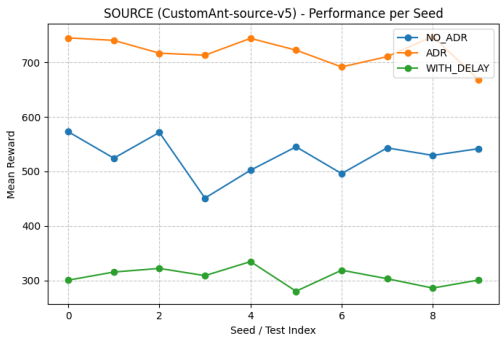

Figure 4. Cumulative Distribution Function

In both the source and target environments, the policy trained with explicit delay randomization exhibits a substantial drop in average performance. This confirms that delay represents a highly disruptive factor for the control task: even small temporal mismatches between action and effect significantly degrade performance.

Overall, ADR adapts only randomization intervals based on performance instead of delay perturbations which are applied using a uniform domain randomization (UDR) strategy, since delays represent discrete and highly disruptive uncertainty.

To better analyze the Ant model, we tested it in different configurations, applying this randomization only to masses and only to frictions to understand which parameters are most significant for the final reward and to analyze how ADRs on frictions and masses can influence each other; the results are reported in Table 5.

Table 5. Robustness statistics in source and target environments.

Env.	Method	Mean $\pm$ Std
Source	ADR on masses	393.8 ± 26.7
	ADR on frictions	319.4 ± 21.6
Target	ADR on masses	381.4 ± 23.7
	ADR on frictions	327.7 ± 20.4

First, if the masses remain fixed, there is a significant drop in performance with an increase in variance. This means that masses are the dominant factor and without their randomization the ADR loses power. Second, by keeping friction constant, we observed an overall drop in performance, but less than in the previous case, with good ability to adapt to mass shifts in the "target" environment.

4. Noise Analysis

Although domain randomization techniques such as UDR and ADR primarily target variations in dynamics, real-world robotic systems are also subject to imperfections in perception. Sensor noise from encoders can significantly degrade policy performance if not accounted for during training. To evaluate the robustness of our policies, we introduced zero-mean Gaussian noise into the observation space during the test step. The noise was applied independently to each dimension of the state vector with a standard deviation of $\sigma = 0.02$

Table 6. Performance under sensor noise across environments.

Model	Method	Mean $\pm$ Std (Source)	Mean $\pm$ Std (Target)
Hopper	NO_ADR	1735.7 ± 44.7	1416.1 ± 258.5
	ADR	1453.2 ± 24.1	1528.9 ± 85.4
Walker2d	NO_ADR	864.2 ± 165.7	735.8 ± 396.0
	ADR	1146.2 ± 473.0	641.0 ± 400.0
Swimmer	NO_ADR	182.0 ± 3.4	186.8 ± 2.5
	ADR	74.7 ± 2.0	74.3 ± 2.1
Ant-v5	NO_ADR	575.1 ± 216.8	536.0 ± 268.2
	ADR	767.7 ± 169.5	708.6 ± 216.8
	With Delay	307.5 ± 189.5	315.3 ± 204.6
	ADR on masses	319.9 ± 191.4	302.8 ± 216.3
	ADR on frictions	356.4 ± 227.9	371.5 ± 261.3

In Hopper, both ADR and non-ADR policies maintain similar mean rewards compared to the noise-free setting. However, the standard deviation increases substantially, especially for the non-ADR model in the target domain, indicating that while the average performance is preserved, the policy becomes less reliable. ADR mitigates this effect, showing smaller increases in variance and more consistent behavior under noise.

In Walker2d, sensor noise causes a dramatic collapse in both mean performance and stability. Walker2d is highly sensitive to observation perturbations. This environment exposes the limits of ADR when the underlying dynamics is inherently unstable and heavily reliant on precise state estimation.

In Swimmer, performance remains virtually unchanged under noise. Both ADR and non-ADR models retain their original mean and low variance. This confirms that Swimmer's dynamics is simple and robust to perceptual noise, making it a poor benchmark for evaluating sensor robustness.

In Ant-5, ADR-trained models maintain high average rewards under noise, with only moderate increases in variance. In contrast, models trained with fixed dynamics (e.g., fixed masses or frictions) suffer significant drops in performance and large increases in variability. The UDR-delay variant remains consistently poor for all conditions. These results confirm that ADR provides meaningful robustness in high-dimensional, nonlinear environments, while fixed-dynamics policies are particularly vulnerable to sensor noise.

4.1. Summary

In general, sensor noise amplifies instability in environments with complex dynamics and high sensitivity to observation accuracy. ADR consistently improves robustness compared to non-ADR baselines, especially in Hopper and Ant. However, in environments like Walker2d, where control is tightly coupled to precise state feedback, even ADR-trained policies degrade sharply. Variance emerges as a key indicator of robustness under noise, often more revealing than mean reward alone.

5. Conclusions

In this work, we explored Adaptive Domain Randomization as a practical and efficient strategy to improve robustness and sim-to-real transfer in reinforcement learning. We proposed a simplified, synchronous ADR formulation that progressively adapts the randomization ranges of the environment parameters based on agent performance, without relying on separate buffers or boundary-specific evaluations. This design significantly reduces algorithmic complexity while enabling simultaneous variation of multiple physical parameters and action delays.

Experimental results for several MuJoCo locomotion environments highlight that the effects of ADR strongly depend on the structure and complexity of the underlying system. In fragile and underactuated environments such as Hopper, ADR consistently trades peak source-domain performance for improved stability, reduced variance, and stronger worst-case behavior under domain shift. In contrast, in highly redundant and nonlinear environments such as Ant, ADR not only enhances robustness, but also improves overall learning outcomes, likely by preventing premature convergence to suboptimal local minima through increased environmental diversity during training.

In simpler environments, such as Walker2d and Swimmer, the benefits of ADR are less pronounced. These results emphasize that ADR is not a universally optimal solution, but rather a targeted robustness mechanism whose effectiveness emerges in environments characterized by high sensitivity to dynamics, redundancy, or significant sim-to-real mismatch.

Additional experiments with sensor noise further demonstrate that ADR-trained policies generally exhibit improved resilience to observation perturbations, particularly in high-dimensional and complex environments. However, tasks that rely heavily on precise state estimation remain challenging, even under adaptive randomization.

Overall, this work shows that a lightweight and performance-driven ADR strategy can provide meaningful robustness benefits with minimal tuning and computational overhead. These findings support the use of simplified ADR formulations as practical tools in improving policy reliability and sim-to-real transfer in continuous-control reinforcement learning.

References

[1] Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. *arXiv*, 2016. 4

[2] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. Rl^2: Fast reinforcement learning via slow reinforcement learning. *arXiv*, 2016. 3

[3] OpenAI et al. Solving rubik's cube with a robot hand. *arXiv preprint*, arXiv:1910.07113, 2019. 1, 2

[4] OpenAI, Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Jozefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *arXiv*, 2019. 1